

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Transforming 3D Models
Source cue	- 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through

Professor-style explanation	The slide 'Transforming 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Transforming 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Transforming 3D Models' before rendering begins?

Page 3	3.1 Mathematical Foundations
Source cue	3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL
Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about 3.1 Mathematical Foundations. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.1 Mathematical Foundations'?

Page 4	3.1 Mathematical Foundations
Source cue	Vector and Matrix Calculus
Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Vector and Matrix Calculus. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Vector and Matrix Calculus. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.1 Mathematical Foundations'?

Page 5	Vertices as Vectors
Source cue	- Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2$, $\mathbb{R}^3$, $\mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$

Professor-style explanation	The slide 'Vertices as Vectors' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of R^2 , R^3 , R^4 / - In computer graphics mostly R^3 or R^4 vectors used / - Vertices of a polygon are represented as column vectors $x \cdot v$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Vertices as Vectors. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of R^2 , R^3 , R^4 / - In computer graphics mostly R^3 or R^4 vectors used / - Vertices of a polygon are represented as column vectors $x \cdot v$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Vertices as Vectors'?

Page 6	Transformations as Matrices
Source cue	- Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$
Professor-style explanation	The slide 'Transformations as Matrices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Transformations as Matrices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformations as Matrices'?

Page 7	Matrix Vector Multiplication
Source cue	- Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B:
Professor-style explanation	The slide 'Matrix Vector Multiplication' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Matrix Vector Multiplication. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Vector Multiplication'?

Page 8	3.2 Affine Transformations
Source cue	Translation, Rotation, Scaling, and more
Professor-style explanation	The slide '3.2 Affine Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Translation, Rotation, Scaling, and more. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.2 Affine Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Translation, Rotation, Scaling, and more. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.2 Affine Transformations'?
Page 9	Translations - Moving 3D Models
Source cue	- Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / T_d , $d = [x \ y \ z]^T$
Professor-style explanation	The slide 'Translations - Moving 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / T_d , $d = [x \ y \ z]^T$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about Translations - Moving 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a translation $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $T(d, d, d)$, $d = x, y, z$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Translations - Moving 3D Models' before rendering begins?

Page 10	Translation Properties
Source cue	- Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / x, y, z
Professor-style explanation	The slide 'Translation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / x, y, z . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Translation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / x, y, z . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Translation Properties'?

Page 11	Scaling - Resizing 3D Models
Source cue	- Matrix representation of a scaling $\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $S(s, s, s)$, $s = x, y, z$

Professor-style explanation	The slide 'Scaling - Resizing 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a scaling $\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $s = 1/x y z$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Scaling - Resizing 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a scaling $\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $s = 1/x y z$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Scaling - Resizing 3D Models' before rendering begins?

Page 12	Scaling Properties
Source cue	- Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$
Professor-style explanation	The slide 'Scaling Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Scaling Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Scaling Properties'?

Page 13	Rotation - Changing Orientation of 3D Models 1/2
Source cue	- Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$
Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 1/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 1/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 1/2' before rendering begins?

Page 14	Rotation - Changing Orientation of 3D Models 2/2
Source cue	- Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x

Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 2/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 2/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 2/2' before rendering begins?
Page 15	Rotation Properties
Source cue	- Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$

Professor-style explanation	The slide 'Rotation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $R \cdot 0 = R \cdot 0 = R \cdot 0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rotation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $R \cdot 0 = R \cdot 0 = R \cdot 0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Properties'?
Page 16	Mirroring - Reflecting 3D Models on a Plane
Source cue	- Matrix representation of mirroring along x-y plane / $1 \ 0 \ 0 \ 0 / 0 \ 1 \ 0 \ 0 / M = / 0 \ 0 \ -1 \ 0$
Professor-style explanation	The slide 'Mirroring - Reflecting 3D Models on a Plane' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of mirroring along x-y plane / $1 \ 0 \ 0 \ 0 / 0 \ 1 \ 0 \ 0 / M = / 0 \ 0 \ -1 \ 0$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Professor-style explanation	The slide '3.3 Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Concatenation of several transformation steps. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.3 Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Concatenation of several transformation steps. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.3 Composition of Transformations'?

Page 19	Composition of Transformations
Source cue	- Composition of transformations T is done / by multiplying the respective transformation matrices / $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))$ / $1 \ 2 \ n-1 \ n \ 1 \ 2 \ n-1 \ n$ / - Interpretation: apply first transformation T , then T , ..., and finally T
Professor-style explanation	The slide 'Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Composition of transformations T is done / by multiplying the respective transformation matrices / $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))$ / $1 \ 2 \ n-1 \ n \ 1 \ 2 \ n-1 \ n$ / - Interpretation: apply first transformation T , then T , ..., and finally T . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Composition of transformations T is done / by multiplying the respective transformation matrices / - $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots \cdot (T \cdot (T \cdot v))))))$ / 1 2 n-1 n 1 2 n-1 n / - Interpretation: apply first transformation T , then T , ..., and finally T . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Composition of Transformations'?

Page 20	Reference Points 1/2
Source cue	- Rotation and scaling happen in relation to origin (reference point)
Professor-style explanation	The slide 'Reference Points 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Rotation and scaling happen in relation to origin (reference point). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Reference Points 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Rotation and scaling happen in relation to origin (reference point). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reference Points 1/2' into a causal sentence instead of repeating the slide title?

Page 21	Reference Points 2/2
Source cue	- In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M

Professor-style explanation	The slide 'Reference Points 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Reference Points 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Reference Points 2/2'?
Page 22	Rotation Around Arbitrary Axis
Source cue	- Rotation around any axis / - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence

Professor-style explanation	The slide 'Rotation Around Arbitrary Axis' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Rotation around any axis / - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rotation Around Arbitrary Axis. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Rotation around any axis / - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Around Arbitrary Axis'?

Page 23	Reminder: Vector Calculus
Source cue	- Length of a vector is expressed as follows $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $v = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows
Professor-style explanation	The slide 'Reminder: Vector Calculus' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Length of a vector is expressed as follows $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $v = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Reminder: Vector Calculus. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Length of a vector is expressed as follows $ v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $ v = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reminder: Vector Calculus'?

Page 24	Reminder: Cross Product (= vector product)
Source cue	- Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows $u \times v = \begin{vmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{vmatrix} u_x & u_y \\ v_x & v_y \end{vmatrix} \mathbf{z} - \begin{vmatrix} u_x & u_z \\ v_x & v_z \end{vmatrix} \mathbf{y} + \begin{vmatrix} u_y & u_z \\ v_y & v_z \end{vmatrix} \mathbf{x}$ / $u \times v = -v \times u$
Professor-style explanation	The slide 'Reminder: Cross Product (= vector product)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows $u \times v = \begin{vmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{vmatrix} u_x & u_y \\ v_x & v_y \end{vmatrix} \mathbf{z} - \begin{vmatrix} u_x & u_z \\ v_x & v_z \end{vmatrix} \mathbf{y} + \begin{vmatrix} u_y & u_z \\ v_y & v_z \end{vmatrix} \mathbf{x}$ / $u \times v = -v \times u$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Reminder: Cross Product (= vector product). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows $u \times v = \begin{vmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{vmatrix} u_x & u_y \\ v_x & v_y \end{vmatrix} \mathbf{z} - \begin{vmatrix} u_x & u_z \\ v_x & v_z \end{vmatrix} \mathbf{y} + \begin{vmatrix} u_y & u_z \\ v_y & v_z \end{vmatrix} \mathbf{x}$ / $u \times v = -v \times u$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reminder: Cross Product (= vector product)' into a causal sentence instead of repeating the slide title?

Page 25	Example - Transforming Directed Line Segment 1/7
Source cue	- Example transformation task / - Given line segments P_1P_2 and P_1P_3 / 1 2 1 3 / - Transform it into the yz -plane so that P_1P_2 lies on the z -axis / 1 2

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 1/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Example transformation task / - Given line segments P and P / 1 2 1 3 / - Transform it into the yz-plane so that P lies on the z-axis / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 1/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Example transformation task / - Given line segments P and P / 1 2 1 3 / - Transform it into the yz-plane so that P lies on the z-axis / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 1/7'?
Page 26	Example - Transforming Directed Line Segment 2/7
Source cue	- Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y -axis so that P lies in the yz -plane / 1 2
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 2/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y-axis so that P lies in the yz-plane / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example - Transforming Directed Line Segment 2/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Approach 1: Determine rotation angles for rotation sequence / 1 1 1 / - Move $P = x, y, z$ into the origin / 1 1 1 / - Rotate around the y-axis so that P lies in the yz-plane / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 2/7'?

Page 27	Example - Transforming Directed Line Segment 3/7
Source cue	- Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 3/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 3/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 3/7'?

Page 28	Example - Transforming Directed Line Segment 4/7
Source cue	- Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z''

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 4/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z'' . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 4/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z'' . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 4/7'?
Page 29	Example - Transforming Directed Line Segment 5/7
Source cue	- Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y■■■■$ / - Matrix ingredients
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 5/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y■■■■$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example - Transforming Directed Line Segment 5/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y \blacksquare \blacksquare \blacksquare$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 5/7'?

Page 30	Example - Transforming Directed Line Segment 6/7
Source cue	- Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis)
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 6/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 6/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 6/7'?

Page 31	Example - Transforming Directed Line Segment 7/7
Source cue	- Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2 , and P_3 / $P_1 \times P_2$ / $R = \frac{P_1 \times P_2}{ P_1 \times P_2 }$ / $T = \begin{bmatrix} 1 & 3 & 1 & 2 \end{bmatrix}$
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 7/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2 , and P_3 / $P_1 \times P_2$ / $R = \frac{P_1 \times P_2}{ P_1 \times P_2 }$ / $T = \begin{bmatrix} 1 & 3 & 1 & 2 \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 7/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2 , and P_3 / $P_1 \times P_2$ / $R = \frac{P_1 \times P_2}{ P_1 \times P_2 }$ / $T = \begin{bmatrix} 1 & 3 & 1 & 2 \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 7/7'?
Page 32	Example - Direction of Flight Transformation 1/3
Source cue	- Example transformation task / - Given 3D model centered in origin of x -, y -, z -coordinate system / p_1, p_2, p_3 / - Transform it such that / - it is centered in arbitrary point P, and

Professor-style explanation	The slide 'Example - Direction of Flight Transformation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Example transformation task / - Given 3D model centered in origin of x -, y -, z -coordinate system / p p p / - Transform it such that / - it is centered in arbitrary point P , and. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Direction of Flight Transformation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Example transformation task / - Given 3D model centered in origin of x -, y -, z -coordinate system / p p p / - Transform it such that / - it is centered in arbitrary point P , and. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 1/3'?

Page 33	Example - Direction of Flight Transformation 2/3
Source cue	r r r / $1x$ $2x$ $3x$ / - Step 1 - create orthogonal rotation matrix $R = r$ r r / $1y$ $2y$ $3y$ / r r r
Professor-style explanation	The slide 'Example - Direction of Flight Transformation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: r r r / $1x$ $2x$ $3x$ / - Step 1 - create orthogonal rotation matrix $R = r$ r r / $1y$ $2y$ $3y$ / r r r . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example - Direction of Flight Transformation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $\begin{bmatrix} r & r & r \\ 1x & 2x & 3x \end{bmatrix} / \begin{bmatrix} 1y & 2y & 3y \\ r & r & r \end{bmatrix}$ - Step 1 - create orthogonal rotation matrix $R = \begin{bmatrix} r & r & r \\ 1y & 2y & 3y \\ r & r & r \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 2/3'?

Page 34	Example - Direction of Flight Transformation 3/3
Source cue	- Transformation M is described by a translation and an orthogonal matrix $\begin{bmatrix} r & r & r & 0 \\ 1x & 2x & 3x \\ r & r & r & 0 \\ 1y & 2y & 3y \end{bmatrix}$
Professor-style explanation	The slide 'Example - Direction of Flight Transformation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transformation M is described by a translation and an orthogonal matrix $\begin{bmatrix} r & r & r & 0 \\ 1x & 2x & 3x \\ r & r & r & 0 \\ 1y & 2y & 3y \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Direction of Flight Transformation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transformation M is described by a translation and an orthogonal matrix $\begin{bmatrix} r & r & r & 0 \\ 1x & 2x & 3x \\ r & r & r & 0 \\ 1y & 2y & 3y \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 3/3'?
Page 35	3.4 Coordinate System Change
Source cue	Transformation as change of frame of reference

Professor-style explanation	The slide '3.4 Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Transformation as change of frame of reference. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.4 Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Transformation as change of frame of reference. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.4 Coordinate System Change'?

Page 36	Coordinate System Change
Source cue	- Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2
Professor-style explanation	The slide 'Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate System Change'?

Page 37	Local Coordinate Systems
Source cue	- Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)
Professor-style explanation	The slide 'Local Coordinate Systems' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Local Coordinate Systems. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Local Coordinate Systems'?

Page 38	Example 1 - Coordinate System Change 1/2
Source cue	$x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q
Professor-style explanation	The slide 'Example 1 - Coordinate System Change 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 1 - Coordinate System Change 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 1/2'?
Page 39	Example 1 - Coordinate System Change 2/2
Source cue	$y(5)$ / $y(2)$ $y(3)$ / $x(1)$ / 3D Model Transformation / $y(5)$ / $y(2)$ $y(3)$
Professor-style explanation	The slide 'Example 1 - Coordinate System Change 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $y(5)$ / $y(2)$ $y(3)$ / $x(1)$ / 3D Model Transformation / $y(5)$ / $y(2)$ $y(3)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example 1 - Coordinate System Change 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $y \ y \ y \ y \ / \ x \ x \ x \ x \ / \ 3D \ Model \ Transformation \ / \ y(5) \ / \ y(2) \ y(3)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 2/2'?

Page 40	Example 2 - Hierarchical Coordinate Systems 1/2
Source cue	- To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system
Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 41	Example 2 - Hierarchical Coordinate Systems 1/2

Source cue	- Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr
Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 42	3.5 Transformations in OpenGL
Source cue	Implementation with GLM und GLSL
Professor-style explanation	The slide '3.5 Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Implementation with GLM und GLSL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about 3.5 Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Implementation with GLM und GLSL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.5 Transformations in OpenGL'?

Page 43	Geometric Transformations in OpenGL
Source cue	- Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements
Professor-style explanation	The slide 'Geometric Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Geometric Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Geometric Transformations in OpenGL'?
Page 44	OpenGL Transformation Pipeline

Source cue	- Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline
Professor-style explanation	The slide 'OpenGL Transformation Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Transformation Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Transformation Pipeline'?
Page 45	Example - Translation and Scaling
Source cue	- Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized

Professor-style explanation	The slide 'Example - Translation and Scaling' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example - Translation and Scaling. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example - Translation and Scaling' into a causal sentence instead of repeating the slide title?

Page 46	Matrix Specification
Source cue	- Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;
Professor-style explanation	The slide 'Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Specification'?

Page 47	Example - Portico Modeling 1/5
Source cue	- Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,
Professor-style explanation	The slide 'Example - Portico Modeling 1/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 1/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 1/5' before rendering begins?

Page 48	Example - Portico Modeling 2/5
Source cue	- Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface

Professor-style explanation	The slide 'Example - Portico Modeling 2/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing a cylinder closed at top and bottom - Each column should "stand" in the y-axis - Each column should have a lid surface at the top and bottom - Disk primitive is embedded in the xy-plane - Cylinder primitive consists only of lateral surface. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 2/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing a cylinder closed at top and bottom - Each column should "stand" in the y-axis - Each column should have a lid surface at the top and bottom - Disk primitive is embedded in the xy-plane - Cylinder primitive consists only of lateral surface. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 2/5' before rendering begins?
Page 49	Example - Portico Modeling 3/5
Source cue	- Drawing a single column - Composition of 5 cylinder objects - Rotation into y-axis to match orientation - Positioning of the parts by translation along the y-axis - Incremental translation around the partial column height

Professor-style explanation	The slide 'Example - Portico Modeling 3/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 3/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 3/5' before rendering begins?

Page 50	Example - Portico Modeling 4/5
Source cue	- Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N , radius R of the columns as well as floor radius R adjustable
Professor-style explanation	The slide 'Example - Portico Modeling 4/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N , radius R of the columns as well as floor radius R adjustable. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about Example - Portico Modeling 4/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 4/5' before rendering begins?

Page 51	Example - Portico Modeling 5/5
Source cue	- Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof
Professor-style explanation	The slide 'Example - Portico Modeling 5/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 5/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 5/5' before rendering begins?

Page 52	by applying affine transformations to their vertices
Source cue	- 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication

Professor-style explanation	The slide 'by applying affine transformations to their vertices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about by applying affine transformations to their vertices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'by applying affine transformations to their vertices'?
Page 53	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.

Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Professor-style explanation	The slide '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' into a causal sentence instead of repeating the slide title?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.